
VILA-STEM 8

ENGINEERING JOURNAL

MADE FOR THE 2026 WRO

FUTURE ENGINEERS COMPETITION

LAST UPDATED: SEPTEMBER 1ST 2026

THE INFORMATION IN THIS DOCUMENT CAN ALSO BE FOUND IN THE OFFICIAL
REPOSITORY AT

[HTTPS://GITHUB.COM/VILA-STEM/WRO_FUTURE_ENGINEERS_2026](https://github.com/VILA-STEM/WRO_FUTURE_ENGINEERS_2026)

Table of Contents

1	Introduction	4
2	Mobility management	6
2.1	Chassis design	6
2.2	How we arrived at this design	7
2.3	Drive system	7
2.4	Steering system	8
3	Sensors and electricity	9
3.1	List of sensors and electrical components	10
3.1.1	ESP32 S3	10
3.1.2	HUSKYLENS 2	11
3.1.3	L298N MOTOR DRIVER	12
3.1.4	MOTOR	13
3.1.5	TF-LUNA	13
3.1.6	ADAFRUIT TCS34725	14
3.1.7	MG995 SERVOMOTOR	15
3.1.8	LM2596 CONVERTERS	16
3.1.9	SWITCH	17
3.1.10	BUTTON	17
3.2	Electrical diagram	18
3.3	Connection type diagram	19
3.4	Power consumption	20
4	Code	21
4.1	open-challenge.ino	23
4.1.1	Introduction to the code	23
4.1.2	Code structure	23
4.1.3	Algorithm design	23
4.1.4	Bugs and errors found in the code	25
4.1.5	Functions	26
4.1.6	Constants	26
4.1.7	Variables	27

4.1.8	Functions documentation	28
4.1.9	Source code	30
4.2	obstacle-challenge.ino	42
4.2.1	Introduction to the code	42
4.2.2	Code structure	42
4.2.3	Algorithm design	42
4.2.4	Bugs and errors found in the code	45
4.2.5	Functions	45
4.2.6	Constants	46
4.2.7	Variables	47
4.2.8	Functions documentation	48
4.2.9	Source code	51
5	Photos	65
5.1	Robot photos	65
5.2	Team photos	68
6	Videos	69

License

The code in this document is licensed under the GNU General Public License v3.0.
Everything not under the GNU General Public License v3.0 in this document
(including photographs, diagrams, etc) is under the Creative Commons
Attribution Share Alike 4.0 International license

Introduction

Vila-stem 8 is the Future Engineers team of the Vila-Stem educational robotics cultural association, located in Vila-real, Spain. Some members of this team have already participated in the WRO in different categories in previous years, including this year, while for others it is their first time competing, but we are all equally excited to participate in the Future Engineers category of WRO 2026.

Our first competition this season was the regional WRO in Dénia, where we won first place and qualified for the national final.

Now we are heading to the national final of WRO Spain 2026 in Andorra, eager to present our project.

Throughout this document, you will find an explanation of our robot and links to other documents that explain the different sections in more detail.

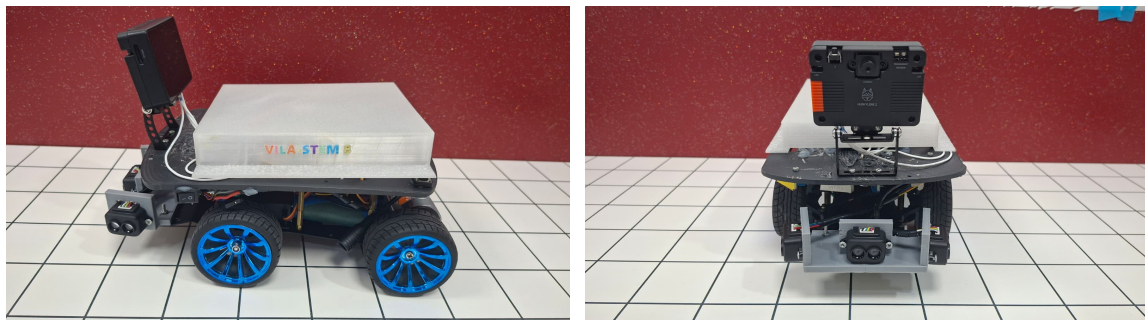
To comply with the competition rules, we have also included a Journal of Engineering in PDF format. This document is a compilation of the information explained throughout this repository.



Figure 1.1: Team photo with a LEGO-like style

Mobility management

2.1 Chassis design



(a) Left image of the robot

(b) Right image of the robot

Figure 2.1: Multiple images of the robot

The lower chassis base comes from a development kit we acquired, so we simply had to assemble it. This kit provided the wheels, motor, and steering system, which simplified assembly. We designed a top platform on top of this kit, where the components are installed, as well as another set of parts to mount the components.

Purchasing this kit allowed us to have a metal chassis that better withstands the robot's stresses. This freed us up to design, using plastic and 3D printing, those parts that wouldn't be subjected to heavy loads.

2.2 How we arrived at this design

Initially, the idea was to mount all the components on the lower chassis, but the lack of space and the desire for a more aesthetically pleasing design led us to install a top platform. This platform saves space and also provides easier access to the connections. We ruled out increasing the robot's height because this keeps the center of gravity low and maintains its compact size.

This year we also aimed for a more aesthetically pleasing robot, which is why the design includes, for example, a decorative top cover and insulated cable covers, among other improvements.

2.3 Drive system

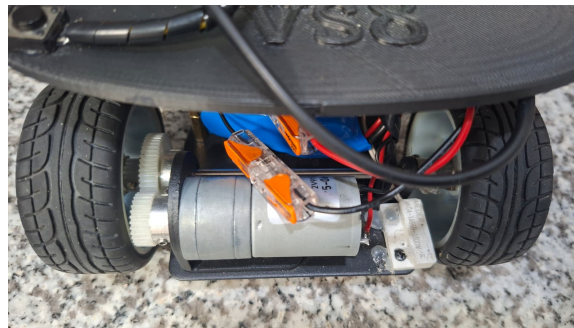


Figure 2.2: Image of the motor mechanism. It can be seen the motor, the wheel axel and the gears

In our robot, the rear wheels are the drive wheels. There is a single axle connecting both wheels, and a gear connects them directly to the motor, without using any differential mechanism. This causes both wheels to always rotate at the same speed.

Although, theoretically, this mechanism could cause the wheels to slip on curves, since the path of travel is different for each wheel in this case, we chose this system because it seemed sufficient for our robot's needs and is easier to install and maintain. A more complex mechanism would have added an extra difficulty that, in the repeated tests we conducted, we didn't find necessary to implement. Any slippage does not affect the robot's proper functioning.

During assembly, we encountered some problems. Some gears and parts didn't fit together, and it took us a while to get them all aligned, but we eventually managed.

2.4 Steering system

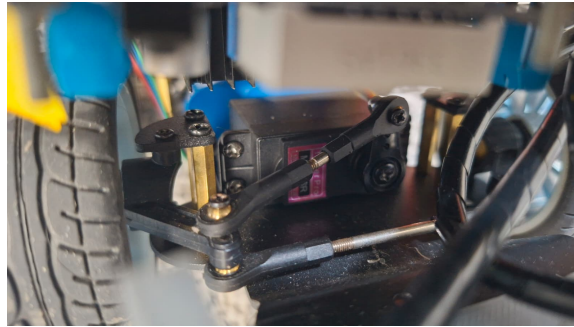


Figure 2.3: Image of the servo mechanism

The steering system is the typical pivot axle (like a wagon): the entire front axle rotates on a central pin moved by the servo. It's the most common system, but it works just as well and is inexpensive, so it was the logical choice for us.

Here, too, we had problems, especially during assembly: some axle parts broke, and we had to replace them with identical ones. There were also parts that were difficult to fit, some because they weren't very well made and others because we got a little confused during assembly.

Physically, there isn't a stop that directly limits the servo's movement. However, the code itself includes limitations that would prevent the wheels from over-rotating and hitting the sides of the chassis.

3

CHAPTER

Sensors and electricity

The robot requires components for both control and power, as well as for obtaining the necessary information from the environment. This section will cover these components, why of their selection, and connection.

INTENTIONALLY BLANK

3.1 List of sensors and electrical components

3.1.1 ESP32 S3



Figure 3.1: Image of the ESP32 S3 microcontroller

- **WHY:** We chose a variant of the ESP32 S3 that includes a camera connection port as the camera data processor. This microcontroller was selected because, being the same type as the main microcontroller, we could work more efficiently with both, only needing to become familiar with one variant. We have two microcontrollers because analyzing camera data is a very resource-intensive process, and therefore it was better to dedicate a microcontroller to it. We selected the OV5640 from several cameras already available in the classroom because it provided the best view of the track after testing all of them.
- **LOCATION:** The microcontroller is located on the upper platform, beneath the top trim cover. The camera is mounted on a front bracket and connected to the microcontroller by a cable.
- **VOLTAGE:** 3.3V (can be powered via a 5V pin connected to a linear regulator)
- **CONNECTION TYPE:** UART communication

3.1.2 HUSKYLENS 2



Figure 3.2: Image of the Huskylens 2

- **WHY:** For much of the robot's development, we tried using an ESP32 S3 CAM with a special lens. We originally chose this option because we thought it would be simpler than using a Huskylens 2, and because our experience with the Huskylens 1 had already been complicated. However, as development progressed, we realized it was actually much more complex. With the ESP32 S3 CAM, we had to manually calibrate several values per code for it to detect colors, and the detection algorithms weren't very effective. In contrast, after some tests with the Huskylens 2, we saw that its ability to be calibrated using the built-in screen and button, along with its improved color detection algorithms, led us to choose it. We had to adapt part of the obstacle code to this new sensor.
- **LOCATION:** It is located on the front of the robot, it has an special stand that keeps it above the robot and looking with a small angle down.
- **VOLTAGE:** 5V (can also be powered via 3.3V)
- **CONNECTION TYPE:** I2C

3.1.3 L298N MOTOR DRIVER

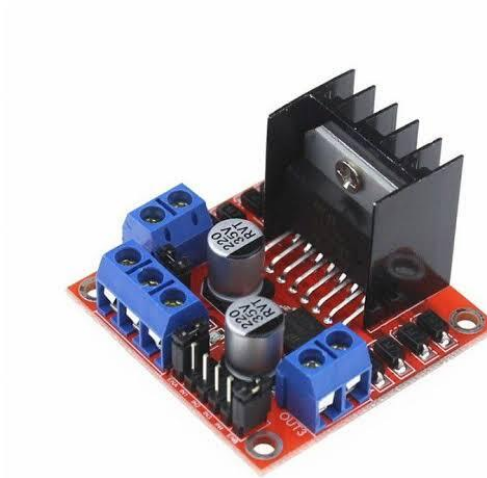


Figure 3.3: Image of the L298N motor driver

- **WHY:** We chose this motor driver because we already had it in the classroom, it generally delivers good results, and it's inexpensive and readily available should we need replacements. In the future, we would like to test the TB6612FNG, as we know it also offers good or even better results and takes up less space. In any case, the L298 motor has not presented any problems in all the tests performed and is easy to operate.
- **LOCATION:** It is located on the underside of the robot's upper platform.
- **MOTOR VOLTAGE:** 12V
- **LOGIC VOLTAGE:** 5V (In our case, it is obtained through a linear regulator from the 12V motor voltage)
- **CONNECTION TYPE:** PWM (speed control), 2 pins (to control the motor's direction or brake)

3.1.4 MOTOR



Figure 3.4: Image of the 12V DC motor

- **WHY:** This motor is a 12V DC motor. Initially, we tested it because it was the one we already had in class, and we wanted to avoid buying new motors if the one we already had was working. It is true that this motor is somewhat bulky compared to other alternatives, but in previous, heavier projects, it never presented any torque or power problems, and the robot was already planned to be somewhat bulky, so we thought it would be a good option. The robot's speed isn't a major concern for us, as the time taken to complete the challenges is low on the tie-breaking priority list in the rules.
- **LOCATION:** On the back of the robot's lower platform.
- **VOLTAGE:** 12V

3.1.5 TF-LUNA



Figure 3.5: Image of the TF-Luna sensor

- **WHY:** To complete the missions, it's essential to know the robot's position relative to the walls. This requires considering the robot's distance from the walls. There are several types of distance sensors: On the one hand, ultrasonic sensors are quite simple, inexpen-

sive, and easy to find, but they offer low accuracy and vary significantly over long distances, which caused us considerable problems in this same challenge last year. Therefore, we quickly ruled them out. Another option is to use Time-of-Flight (ToF) sensors. These sensors emit a beam of light and calculate the time between its emission, reflection, and return, allowing for precise determination of the distance to the reflected object, since the speed of light is constant. We noticed that many WRO projects use a rotating ToF sensor called LiDAR. After conducting some tests with a LiDAR we had and reading reviews from others who had worked with this sensor, we found it difficult to use, so we ruled out LiDAR. Finally, we selected fixed ToF sensors, the TF-Luna. The TF-Luna is relatively inexpensive, and after our initial testing, we found it to be quite accurate. Specifically, we tested it by measuring multiple distances between 5 cm and 3 m (dimensions designed for the Future Engineers challenge), and in every case, the distance obtained with the TF-Luna perfectly matched the physical measurement taken with a meter stick.

- **LOCATION:** We have three sensors, one on each front side and one at the front. The side sensors are angled slightly forward. These sensors are mounted on a specially designed bracket and anchored to the lower platform.
- **VOLTAGE:** 5V
- **CONNECTION TYPE:** I2C

3.1.6 ADAFRUIT TCS34725

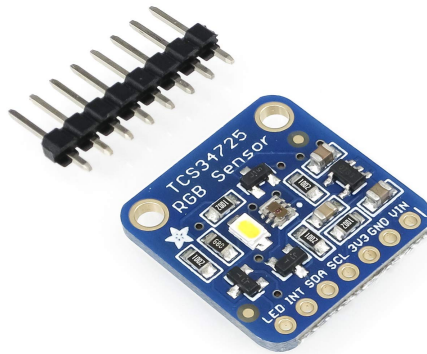


Figure 3.6: Image of the Adafruit TCS34725 color sensor

- **WHY:** To determine the robot's position on the board and the number of sections traversed, we use orange lines conveniently placed at the corners of the board. We measure the robot's movement over these lines using this color sensor. We chose this sensor after finding positive online reviews and because it offered an I2C connection. Before using it, we calibrated it, and after testing, we found it to be reliable and accurate, so we kept it.
- **LOCATION:** The lower platform has a hole facing the floor for the color sensor.
- **VOLTAGE:** 3.3V

- **CONNECTION TYPE:** I2C

3.1.7 MG995 SERVOMOTOR



Figure 3.7: Image of the MG995 servomotor

- **WHY:** A servomotor is necessary to control the steering of the front wheels. This servomotor was chosen because it offers higher torque (9.4 kg/cm) compared to other popular options like the SG90 (1.8 kg/cm), while maintaining a relatively low price and good availability. Price is a particularly important factor in this case, as our experience competing in this category shows that servomotors tend to wear out considerably during training and competition, requiring frequent replacement. Therefore, we prioritized a good price.
- **LOCATION:** On the lower platform.
- **VOLTAGE:** 5V
- **CONNECTION TYPE:** I2C

3.1.8 LM2596 CONVERTERS

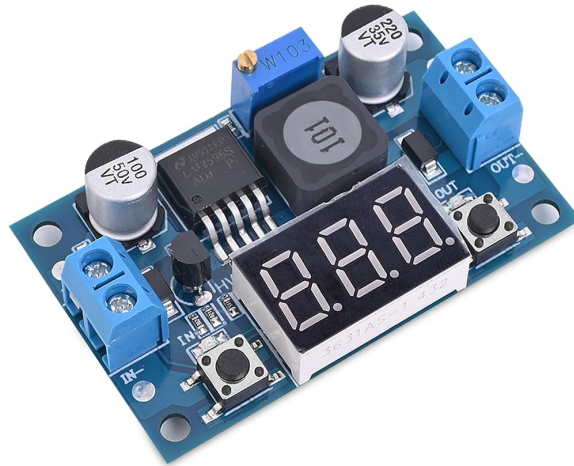


Figure 3.8: Image of the LM2596 converter

- WHY:** The 12V we get from the battery needs to be converted to 5V and 3.3V for the other components to function. This requires a converter. There are mainly two types: linear and switched-mode. Linear converters, like the LM7805, have the problem of being energy inefficient and generating a lot of heat when converting excess voltage. On the other hand, switched-mode converters offer higher efficiency and generate less heat. Therefore, we selected a switched-mode voltage converter. The advantage of choosing this particular version of the LM2596 is that it is adjustable, which allows us to have several identical converters in stock to meet the robot's needs. It also has a display that shows the output voltage in real time, useful for monitoring the power status without needing a multimeter. We have 2 12V to 5V converters because after installing the Huskylens 2 we saw that amount of current it used exceeded the limits and made the voltage drop to the point that all the components connected to the only 5V converter stopped working, so we added one 12V to 5V only for the Huskylens 2 and another for all the rest of components.
- LOCATION:** 2 are located on the underside of the robot's upper platform and the Huskylens 2 5V converter is located on top of the robot's upper platform.

3.1.9 SWITCH



Figure 3.9: Image of main power switch

- **WHY:** According to regulations, a switch is required to cut off or activate the robot's power.
- **LOCATION:** On the side of the robot's upper platform.

3.1.10 BUTTON

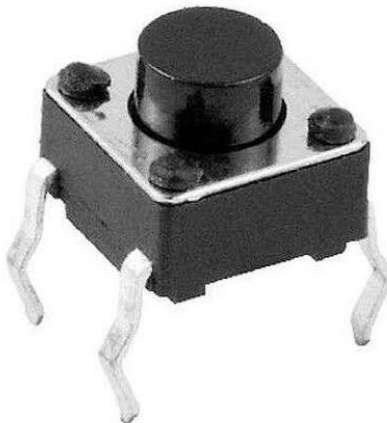


Figure 3.10: Image of the start button

- **WHY:** According to regulations, a separate button is required to activate the robot's execution of the challenge.
- **LOCATION:** On the robot's upper platform.

3.2 Electrical diagram

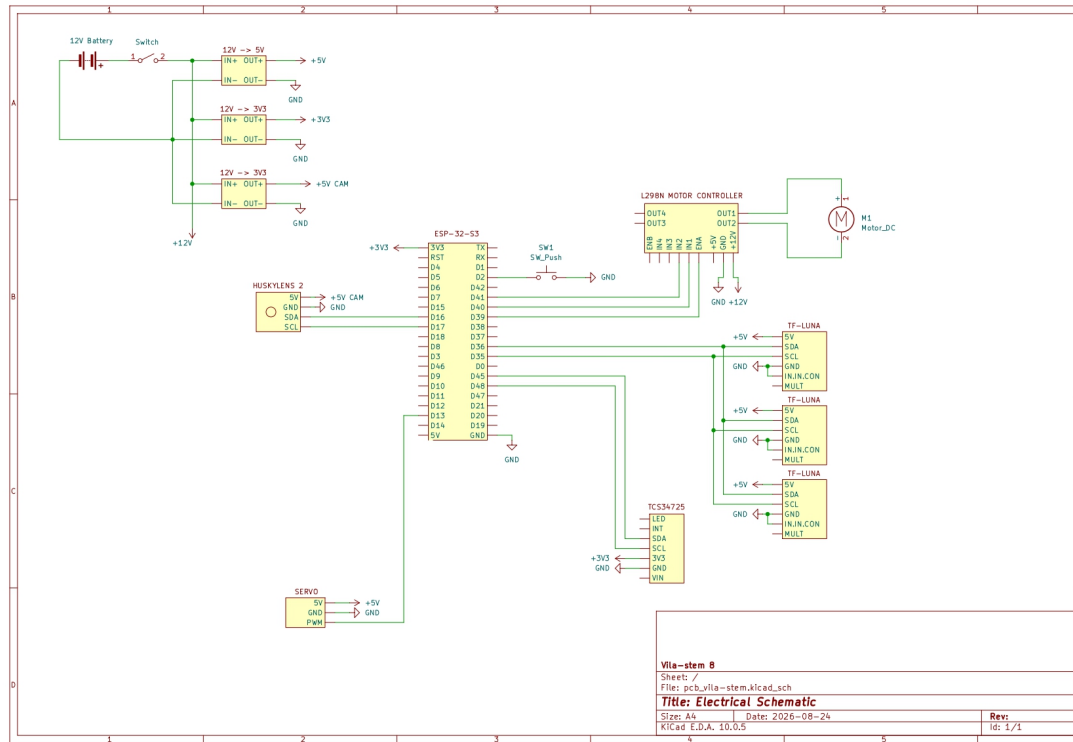


Figure 3.11: Robot electrical diagram showing all the different connections

3.3 Connection type diagram

This diagram (fig. 3.12) shows the connection type of each component to the main microcontroller. The pink squares represent the microcontroller. The blue octagons are the sensors that send information to the main microcontroller. The yellow circles are the actuators.

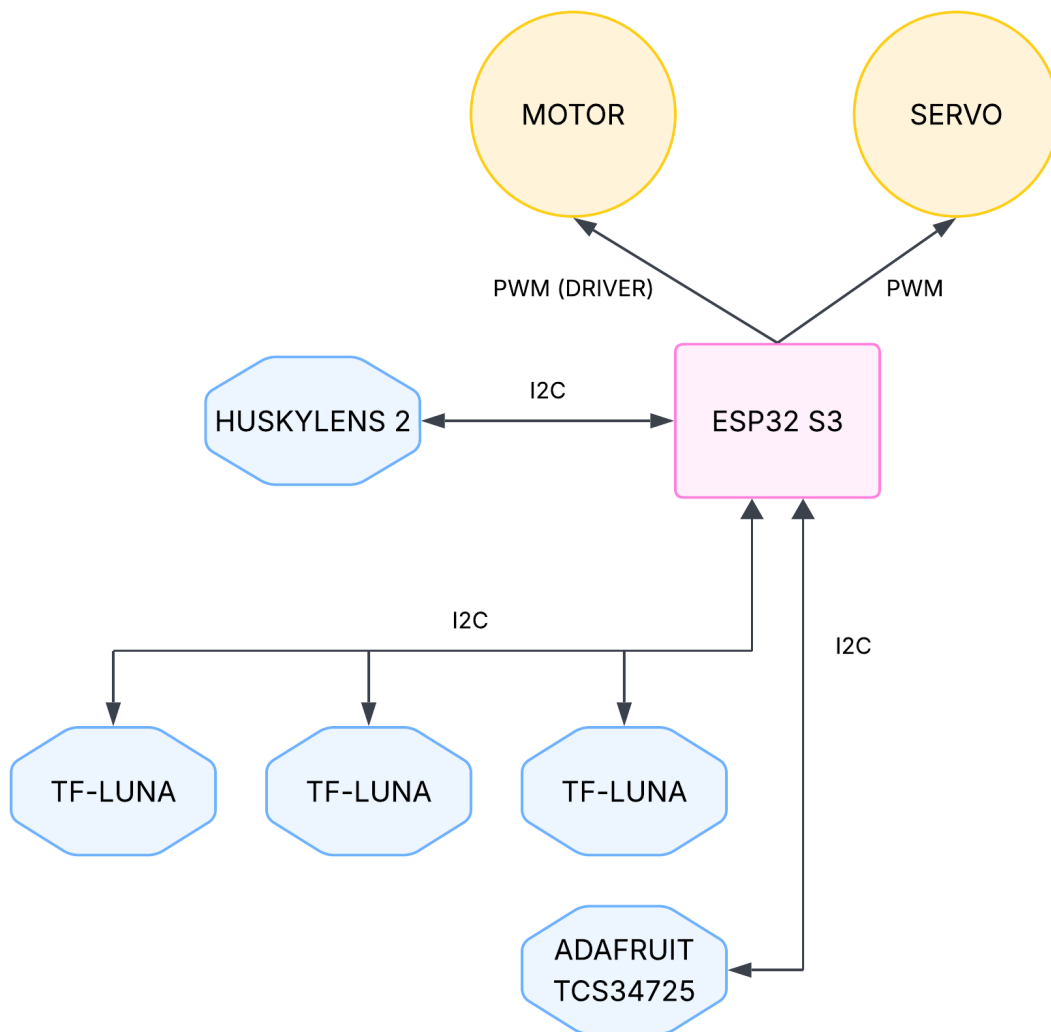


Figure 3.12: This shows the connection type of each component

3.4 Power consumption

COMPONENT	VOLTAGE (V)	CURRENT (mA)
3 * LM2596 Converters	12	50
Servomotor MG995	5	≈ 200 - 1200
Adafruit TCS34725	3.3	10
Motor	12	≈ 300
3 * TF-Luna	5	70
L298N Motor driver	Vm.: 12; Vlog.: 5	36
HuskyLens 2	5	≈ 300 - 600
ESP32 S3	3.3	≈ 50

Table 3.1: This table shows the consumption of each component

4

CHAPTER

Code

The following diagram (fig. 4.1) shows which code should be uploaded to the microcontroller in each challenge.

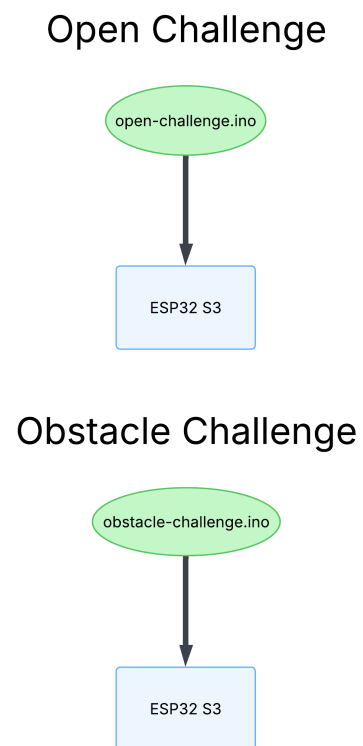


Figure 4.1: Which code should be uploaded to the microcontrollers in each challenge

As we can see there are two programs: one for the Open Challenge and one for the Obstacle Challenge, both uploaded to the only ESP32 S3.

In this chapter we will show how each code works, its functions and the source code.

INTENTIONALLY BLANK

4.1 open-challenge.ino

4.1.1 Introduction to the code

Although at first glance the code for the Open Challenge may seem trivial due to its apparent simplicity compared to the Obstacle Challenge, our goal with this code was not only to create something simple to quickly overcome this challenge, but also to create a foundation upon which to build the Obstacle Challenge code. With this idea in mind, our algorithm design and code structure have been designed to facilitate this transition.

On the other hand, we understand that the Open Challenge should demonstrate the minimum the robot is capable of, which is why we have also prioritized creating robust yet simple code.

4.1.2 Code structure

All the code for this challenge is contained in a single .ino file, named open-challenge.ino. This code is executed in the robot's ESP32 main.

There are no other programs for this challenge.

We have not created any additional libraries or files as required.

4.1.3 Algorithm design

A flowchart of the code's operation can be seen below (fig. 4.2).

As can be seen, as soon as the robot is powered on, it enters standby mode until the Start button is pressed. Meanwhile, as observed in the source code, the robot's various systems and peripherals are initialized (I2C communication, color sensor, servomotor, etc.).

Once the Start button is pressed, the robot enters the loop. We can divide the loop's operation into two parts:

1. Robot Movement
2. Limit Case Checking

4.1.3.1 Robot movement

The strategy here is simple: keep the robot centered between the side walls at all times.

To achieve this, the Luna ToF sensors located on the sides of the robot measure the distance to the right and left. Then, using the difference between these distances, the servomotor is instructed to turn the wheels in the corresponding direction. For example, if the distance to the right is greater than to the left, the robot will turn its wheels to the right, and if the distance to the

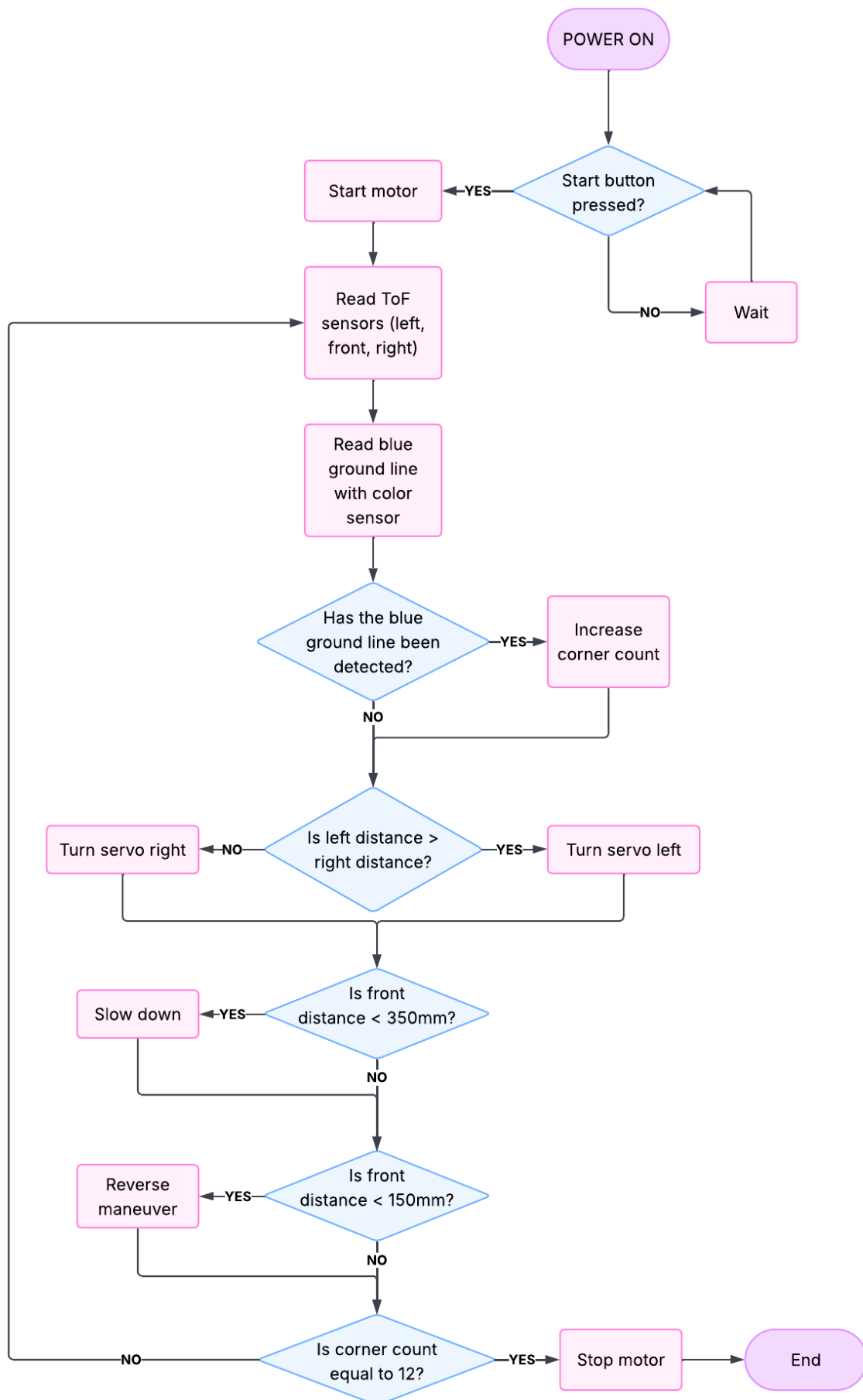


Figure 4.2: Open challenge algorithm flowchart

left is greater than to the right, the wheels will turn to the left. The benefit of using the difference in distances is that it allows the servomotor to turn only slightly if the difference is small, while if the difference is very large, it turns much more, allowing for better cornering.

With this, after testing, we have ensured that regardless of the direction of rotation, the starting position, or the size of the central square, the robot always remains centered.

4.1.3.2 Check of limit cases

The loop also performs certain checks at all times on the following things, as shown in the flowchart:

- If the color sensor facing the ground detects the blue line, then the internal counter `corner_count`, which measures how many corners the robot has passed through, is incremented by one. This tells us how many sections the robot has traversed.
- If the front Luna ToF sensor measures a distance less than 350mm, the motor speed is reduced to avoid a collision. This can sometimes happen when getting too close to walls.
- If the front Luna ToF sensor measures a distance less than 150mm, a collision is practically imminent, and the reversing process is activated. This temporarily disables the robot's movement and makes it reverse to avoid getting stuck.
- If the `corner_counter` is equal to 12, the robot has already completed three laps and must stop in the final section.

4.1.4 Bugs and errors found in the code

Throughout the development of this code, we encountered various problems and challenges that we had to resolve. This, in turn, allowed us to create a more robust code than we originally envisioned.

Initially, some minor problems we had included incorrectly assigning the I2C address to each Luna ToF sensor and missing resistors here and there. But above all, we would highlight the following problems we encountered, as their solution has improved the code's resilience, and we believe other users might encounter similar issues.

- One problem we found was that the Luna ToF sensors, when measuring very long distances (which we considered to be infinity), returned a measurement of 0 mm. On the other hand, when very close to a wall, since they weren't designed to measure such short distances, they also returned some erroneous values, highly variable, and in many cases, also 0 mm. This presented the problem of how to distinguish between 0 at a very long distance and 0 at a very short distance. For this, we used the "strengt" data provided by the Luna ToF sensors themselves. After several tests, we verified that the strength was minimal when it was 0 at infinity and very high when it was 0 at very close range. Therefore,

every time we obtained a value of 0, we performed a second check with the strength to determine what type of 0 it was. If it was 0 at infinity, we set it to 1500 mm.

- Sometimes, when the robot had to make very tight turns, it would pick up a lot of speed and crash into the walls. We originally designed a reversing mechanism that should have solved this problem. However, the frequency and manner of the collisions caused it to take a long time to complete the circuit, and in many cases, the reversing maneuver was ineffective. This was resolved by having the robot reduce its speed when it detects that it is approaching the walls. In many cases, this avoids the collision, and in those where it doesn't, the slower collision usually leaves the robot in a better position to more easily complete the circuit after reversing.

4.1.5 Functions

	Name
void IRAM_ATTR	OdometerInterrupt()
uint16_t	LunaRead16(uint8_t addr, uint8_t reg)
uint16_t	LunaDistance(uint8_t addr)
void	Drive()
void	ServoTurning(signed int _distance_right, signed int _distance_left)
void	setup()
void	loop()

Table 4.1: Each function and its type

4.1.6 Constants

Table 4.2: Each variable and its type

	Name	Value
const uint8_t	LUNA_COUNT Number of Luna time of flight sensors	3
const uint8_t[LUNA_COUNT]	LUNA_ADDR I2C address of each ToF sensor. Right, Left, Front	0x11, 0x12, 0x13
const uint16_t	LUNA_STRENGHT_MIN Minimum strenght needed from the ToF sensor to accept a measurement	20

Continues on next page...

Table 4.2: Each variable and its type (continued)

... comes from previous page		
	Name	Value
const uint16_t	LUNA_CLOSE_STRENGTH Minumum strength needed to consider a 0 distance measurement too close	150
const uint16_t	LUNA_CLOSE_MM Value (mm) used in case of a too close measurement	150
const uint16_t	FRONT_BRAKE_MM Progressive braking start distance (mm) threshold	350
const uint16_t	FRONT_REAR_MM Reverse gear start distance (mm) threshold	150
const int	MOTOR_B_PIN	20
const int	MOTOR_A_PIN	10
const int	ENA_PIN	15
const int	BUTTON_PIN	13
const int	ODOMETER_PIN	15
const int	SDA_PIN	11
const int	SCL_PIN	12
const int	SDA_PIN_COLOR	40
const int	SCL_PIN_COLOR	41
const int	SERVO_PIN	9
const int	BASE_SPEED	120

4.1.7 Variables

Table 4.3: Each variable and its type

	Name
Adafruit_TCS34725	tcs Color sensor TCS34725 object.
Servo	servo Servo object.
int	button_state START button state.
int	corner_count Counts the number of corners the robot has passed. It should go from 0 to 12.
Continues on next page...	

Table 4.3: Each variable and its type (continued)

... comes from previous page	
	Name
long	delay_line NOT IN USE.
int	travel_sense Controls travel sense. 0 = BRAKE, 1 = FORWARD, 2 = BACKWARD.
byte	rear_phase Controls the rear phases. 0 = NORMAL, 1 = STOP, 2 = REVERSE, 3 = RESUME.
long	end_motor_millis Variable to control the additional time added from entering the last section until the motor stops.
long	prev_millis_line_detection Variable to control time and avoid multiple detections of the same line.
long	prev_millis_front Variable to control time and avoid multiple detections of the wall from the front sensor.
unsigned long	rear_phase_until Variable to control time for the reverse system.
long	odo_filter Variable to control time and avoid multiple detections of the same step of the odometer.
int	odo_integral Odometer step count.
int	d_servo How much the servo needs to move.

4.1.8 Functions documentation

4.1.8.1 function OdometerInterrupt

```

1 void IRAM_ATTR OdometerInterrupt()
2

```

Parameters: None

Return: void

Odometer interrupt. It uses the ESP32 interrupt management system so that when the odometer moves, this code is executed.

4.1.8.2 function LunaRead16

```
1      uint16_t LunaRead16(uint8_t addr,uint8_t reg)
2
```

Parameters:

- **addr** I2C address of the ToF Luna that you wish to request
- **reg** Value to be requested. 0x00 for distance. 0x02 for strength

Return: Desired value of the ToF Luna in question

Request the demanded value of the indicated ToF Luna via I2C connectivity.

4.1.8.3 function LunaDistance

```
1      uint16_t LunaDistance(uint8_t addr)
2
```

Parameters:

- **addr** I2C address of the ToF Luna that you wish to request

Return:

- Distance in mm measured
- 0xFFFF if failed to measure

This function returns the distance in mm measured by the requested Luna ToF and also ensures that this measurement makes sense using the strength of the Luna ToF, thus giving consistent results in borderline cases.

4.1.8.4 function Drive

```
1      void Drive()
2
```

Parameters: None

Return: void

The driving and collision avoidance algorithm.

4.1.8.5 function ServoTurning

```
1         void ServoTurning(signed int _distance_right, signed
2         int _distance_left)
```

Parameters:

- **_distance_right** The right distance measured by the ToF Luna sensor
- **_distance_left** The left distance measured by the ToF Luna sensor

Return: void

The servo turning algorithm.

4.1.8.6 function setup

```
1         void setup()
```

Parameters: None

Return: void

This code only runs once when the ESP32 starts up. Initialize all systems and peripherals and wait for the start button to be pressed to start the program.

4.1.8.7 function loop

```
1         void loop()
```

Parameters: None

Return: void

This code only runs all the time.

4.1.9 Source code

```
1  /*****
2  *   FILE NAME:   open-challenge
3  *   *****/
```

```

4  *
5  *  PURPOSE:      This program is the version of the main
6  *                robot code for the Open Challenge of the
7  *                Future Engineers category of the World
8  *                Robot Olympiad.
9  *
10 *
11 *  HOW IT WORKS: The program uses side time-of-flight
12 *                distance sensors to keep the robot
13 *                constantly aligned with the center line
14 *                between the two side walls. Additionally,
15 *                in the event of a frontal collision or
16 *                when approaching a collision, the front
17 *                sensor is used to move backward or reduce
18 *                speed, respectively.
19 *
20 *
21 *  LIBRARIES USED:
22 *  Name                Description
23 *  ----                -
24 *  ESP32Servo          Functions required to control a
25 *                      servomotor with the ESP32
26 *  Wire                Allows I2C communication
27 *  Adafruit_TCS34725   Functions required to use the
28 *                      Adafruit TCS34725 RGB Color Sensor
29 *
30 *
31 *  OTHER PROGRAMS NEEDED TO WORK
32 *  Name                Description
33 *  ----                -
34 *  none
35 *
36 *
37 *  CREATED BY: Vila-Stem 8
38 *
39 *
40 *  LAST MODIFIED: August 31st 2026
41 *
42 *
43 *  REPOSITORY: https://github.com/Vila-Stem/
44 *              WRO_FUTURE_ENGINEERS_2026
45 *
46 *  *****
47 */
48
49 /* INCLUDE LIBRARIES */
50 #include <ESP32Servo.h>
51 #include <Wire.h>
52 #include <Adafruit_TCS34725.h>
53

```



```

91 int travel_sense = 1;          ///< Controls travel sense. 0
    = BRAKE, 1 = FORWARD, 2 = BACKWARD
92 byte rear_phase = 0;          ///< Controls the rear phases.
    0 = NORMAL, 1 = STOP, 2 = REVERSE, 3 = RESUME
93 long end_motor_millis = 0;     ///< Variable to control the
    additional time added from entering the last section until the
    motor stops
94 long prev_millis_line_detection = 0; ///< Variable to control time
    and avoid multiple detections of the same line
95 long prev_millis_front = 0;     ///< Variable to control time
    and avoid multiple detections of the wall from the front sensor
96 unsigned long rear_phase_until = 0; ///< Variable to control time
    for the reverse system
97 long odo_filter = 0;           ///< Variable to control time
    and avoid multiple detections of the same step of the odometer
98 int odo_integral = 0;          ///< Odometer step count
99 int d_servo = 0;              ///< How much the servo needs
    to move

100
101
102 /* FUNCTIONS */
103 /**
104  * Odometer interrupt. It uses the ESP32 interrupt management
105  * system so that when the odometer moves, this code is executed.
106  * @param none
107  * @return void
108  */
109 void IRAM_ATTR OdometerInterrupt(){
110     if ((micros()-odo_filter)>1000) // Avoids double detecting the
        same step
111     {
112         switch (travel_sense)
113         {
114             case 0:
115                 odo_integral++;
116                 break;
117             case 1:
118                 odo_integral++;
119                 break;
120             case 2:
121                 odo_integral--;
122                 break;
123         }
124         odo_filter = micros();
125     }
126 }

127
128
129 /**
130  * Request the demanded value of the indicated ToF Luna via I2C
    connectivity.

```

```

131 * @param addr I2C address of the ToF Luna that you wish to request
132 * @param reg Value to be requested. 0x00 for distance. 0x02 for
    strength
133 * @return Desired value of the ToF Luna in question
134 * @see LunaDistance()
135 */
136 uint16_t LunaRead16(uint8_t addr, uint8_t reg) {
137     Wire.beginTransmission(addr);                                // Begins
    I2C communication with the given address
138     Wire.write(reg);                                            // Tells ToF
    Luna that we desire the given type of information
139     Wire.endTransmission(false);                                // Keeps the
    I2C communication active with the ToF Luna
140     Wire.requestFrom(addr, (uint8_t)2);                        // Requests
    2 bytes data from the ToF Luna. It releases the I2C bus after the
    request.
141     if (Wire.available() < 2) return 0xFFFF;                    // If less
    than 2 bytes retrieved returns 0xFFFF as failed
142     return (uint16_t)(Wire.read() | (Wire.read() << 8)); // Returns
    the data
143 }
144
145
146 /**
147  * This function returns the distance in mm measured by the
    requested
148  * Luna ToF and also ensures that this measurement makes sense using
149  * the strength of the Luna ToF, thus giving consistent results in
150  * borderline cases.
151  * @param addr I2C address of the ToF Luna that you wish to request
152  * @return Distance in mm measured
153  * @return 0xFFFF if failed to measure
154  * @see LunaRead16()
155  */
156 uint16_t LunaDistance(uint8_t addr) {
157     uint16_t d = LunaRead16(addr, 0x00);                        // Request ToF Luna
    distance
158     uint16_t strength = LunaRead16(addr, 0x02);                // Request ToF Luna
    strength
159     if (d == 0xFFFF || strength == 0xFFFF || strength <
    LUNA_STRENGTH_MIN) return 0xFFFF; // Returns 0xFFFF if failed to
    measure or the strength is too low
160     if (d == 0)                                                // The distance may
    be 0 because the sensor is too close to a wall and is saturated,
    or because it measures infinity
161     {
162         if (strength >= LUNA_CLOSE_STRENGTH) return LUNA_CLOSE_MM; //
    If the strength is very high, the sensor is too close to the wall
    .
163         return 1500;                                            //
    If the strength is very low, the sensor measures infinity.

```

```

164     }
165     return d*10;    // The ToF Luna returns cm, but the code needs mm
166 }
167
168
169 /**
170  * The driving and collision avoidance algorithm.
171  * @param none
172  * @return void
173  * @see LunaDistance()
174  * @see ServoTurning()
175  */
176 void Drive()
177 {
178     // Initialize variable for reading the ToF sensors
179     static signed int distance_right, distance_left, distance_front;
180     uint16_t d;
181
182     // These lines read the ToF Luna sensors and save the distance
183     // in the corresponding variable as long as it hasn't failed
184     // reading.
185     d = LunaDistance(LUNA_ADDR[0]);
186     // Right distance sensor
187     if (d != 0xFFFF)
188     {
189         distance_right = d;
190         if (distance_right > 1500) distance_right = 1500;
191         // If the distance is greater than 1500mm, it limits it to
192         // 1500mm.
193     }
194     d = LunaDistance(LUNA_ADDR[1]);
195     // Left distance sensor
196     if (d != 0xFFFF)
197     {
198         distance_left = d;
199         if (distance_left > 1500) distance_left = 1500;
200         // If the distance is greater than 1500mm, it limits it to
201         // 1500mm.
202     }
203     d = LunaDistance(LUNA_ADDR[2]);
204     // Front distance sensor
205     if (d != 0xFFFF)
206     {
207         distance_front = d;
208     }
209
210     // If front sensor detects an obstacle init rear sequence. This
211     // sequence is time based
212     if (distance_front < FRONT_REAR_MM)
213     {
214         if (rear_phase == 0)

```

```

206     {
207         if ((prev_millis_front + 1200) <= millis())    // This if
statement avoids multiple detection of the same wall
208         {
209             prev_millis_front = millis();
210             rear_phase = 1;
211             rear_phase_until = millis() + 250;
212         }
213     }
214 }
215
216 // Rear sequence. This is time based
217 if (rear_phase >= 1)
218 {
219     switch (rear_phase)
220     {
221         case 1:                // Stops the car for 250 mm (by deafult)
222             travel_sense=0;
223             if (millis() >= rear_phase_until){ rear_phase=2;
rear_phase_until = millis() + 550; }
224             break;
225         case 2:
226             travel_sense=2;    // Reverse the car for 550 mm (by deafult)
227             if (millis() >= rear_phase_until){ rear_phase=3;
rear_phase_until = millis() + 50; }
228             break;
229         case 3:                // The robot continues forward.
230             travel_sense=1;
231             rear_phase=0;
232             break;
233     }
234 }
235
236 ServoTurning(distance_right, distance_left);    // Servo turning
algorithm
237
238 // Speed control
239 int base_speed = BASE_SPEED;                    // Base speed of
the robot
240
241 if (distance_front < FRONT_REAR_MM)              // Reduces the
speed before starting the reverse manouver
242 {
243     base_speed = 60;
244 }
245 else if (distance_front < FRONT_BRAKE_MM)        // Reduces the
speed when approaching a frontal wall to avoid a collision or
having to use the reverse manouver
246 {
247     base_speed = 75;
248 }

```

```

249
250   if (d_servo >= 0)
251   {
252       ledcWrite(ENA_PIN, base_speed + d_servo);    // Sends the motor
253       the speed. It is increased when turning
254   }
255   else
256   {
257       ledcWrite(ENA_PIN, base_speed - d_servo);    // Sends the motor
258       the speed. It is increased when turning
259   }
260 }
261
262 /**
263  * The servo turning algorithm.
264  * @param _distance_right The right distance measured by the ToF
265  *   Luna sensor
266  * @param _distance_left The left distance measured by the ToF Luna
267  *   sensor
268  * @return void
269  * @see Drive()
270  */
271 void ServoTurning(signed int _distance_right, signed int
272   _distance_left)
273 {
274     // Initialize needed variables
275     signed int distance_right = _distance_right;
276     signed int distance_left = _distance_left;
277
278     // The servo needs to move a certain amount depending on the
279     // difference of distance measured between the left and
280     // the right ToF Luna sensors
281     d_servo = (distance_right - distance_left)/20;
282
283     // Limits the amount a servo can move in order to avoid collision
284     // of the wheels and the frame
285     if (d_servo > 39) d_servo = 39;
286     if (d_servo < -39) d_servo = -39;
287
288     // In case of reverse, limited steering to avoid flipping
289     if (rear_phase == 2)
290     {
291         if (d_servo > 12) d_servo = 12;
292         if (d_servo < -12) d_servo = -12;
293         servo.write(-d_servo + 85);
294     }
295     else if (travel_sense == 1)
296     {
297         servo.write(d_servo + 85);
298     }
299 }

```

```

294 }
295
296
297 /* SETUP AND LOOP FUNCTIONS */
298
299 /**
300  * This code only runs once when the ESP32 starts up.
301  * Initialize all systems and peripherals and wait for the
302  * start button to be pressed to start the program.
303  * @param none
304  * @return void
305  */
306 void setup() {
307     Serial.begin(115200);                // Starts Serial
308     // for debugging reasons
309     Wire.begin(SDA_PIN, SCL_PIN);        // I2C
310     // communication for ToF Luna
311     Wire.setTimeout(100);
312     Wire.setClock(100000);               // ToF Luna at
313     // 100kHz
314     Wire1.begin(SDA_PIN_COLOR, SCL_PIN_COLOR); // I2C
315     // communication for TCS34725 color sensor
316
317     servo.attach(SERVO_PIN);             // Initialize
318     // Servo
319     servo.write(90);                     // 0 position
320     // Servo
321
322     // Pin Modes
323     pinMode(BUTTON_PIN, INPUT_PULLUP);
324     pinMode(MOTOR_A_PIN, OUTPUT);
325     pinMode(MOTOR_B_PIN, OUTPUT);
326
327     // Stop the motor
328     digitalWrite(MOTOR_A_PIN, LOW);
329     digitalWrite(MOTOR_B_PIN, LOW);
330
331     ledcAttachChannel(ENA_PIN, 1000, 8, 2); // PWM for the
332     // motor. Frequency = 1000, Resolution = 8, Channel = 2
333
334     // FOR DEBUGGING
335     // Checks ToF Luna presence and debug which ones don't work
336     // through the serial port.
337     for (uint8_t i = 0; i < LUNA_COUNT; i++)
338     {
339         Wire.beginTransaction(LUNA_ADDR[i]);
340         if (Wire.endTransmission() == 0)
341         {
342             Serial.print("TF-Luna ");
343             Serial.print(i);
344             Serial.print(" OK (0x");

```

```

337     Serial.print(LUNA_ADDR[i], HEX);
338     Serial.println("");
339 }
340 else
341 {
342     Serial.print("TF-Luna ");
343     Serial.print(i);
344     Serial.print(" NOT found (0x");
345     Serial.print(LUNA_ADDR[i], HEX);
346     Serial.println(")");
347 }
348 }
349
350 // Initialize color sensor. DEBUG trougth the serial port if not
351 // working.
352 if (tcs.begin(0x29, &Wire1))
353 {
354     Serial.println("Found sensor");
355 }
356 else
357 {
358     Serial.println("WARNING: TCS34725 not detected, continuing
359 // without color.");
360 }
361
362 // Initialize odometer
363 //attachInterrupt(ODOMETER_PIN, OdometerInterrupt,CHANGE); // NOT
364 // IN USE
365
366 // Waits for START button to be pressed
367 int countButton = 0;
368 while (countButton <= 10)
369 {
370     button_state = digitalRead(BUTTON_PIN);
371     if (button_state == LOW) countButton++;
372     delay(10);
373 }
374 }
375
376 /**
377  * This code only runs all the time.
378  * @param none
379  * @return void
380  */
381 void loop() {
382     // Gets color temp from the Adafruit TCS34725 color sensor
383     uint16_t r, g, b, c, colorTemp; //
384     // Initialize needed variables
385     tcs.getRawData(&r, &g, &b, &c); //
386     // Gets the RGB and Clear channels

```

```

383 colorTemp = tcs.calculateColorTemperature_dn40(r, g, b, c);    //
    Calculates the color temperature from the RGB channels
384
385 //Serial.print("Color Temp: "); Serial.println(colorTemp, DEC); //
    FOR DEBUGGING. Use it to calibrate the sensor
386
387 // Increases corner count using the lines on the ground and the
    TCS34725 color sensor.
388 if (colorTemp > 3000)
389 {
390     if ((prev_millis_line_detection + 700) <= millis())
391     {
392         prev_millis_line_detection = millis();
393         corner_count++;
394     }
395 }
396
397 // Stops the car when the number of laps is reached (12 corners)
398 if (corner_count >= 12)
399 {
400     if ((end_motor_millis + 1500) <= millis())
401     {
402         travel_sense=0;
403     }
404 }
405 else
406 {
407     end_motor_millis = millis();
408 }
409
410 // Sets the travel sense. 0 = BRAKE, 1 = FORWARD, 2 = BACKWARD
411 switch(travel_sense)
412 {
413     case 0:
414         digitalWrite(MOTOR_A_PIN, LOW);
415         digitalWrite(MOTOR_B_PIN, LOW);
416         break;
417     case 1:
418         //digitalWrite(MOTOR_A_PIN, LOW);    // FOR DEBUGGING.
419         digitalWrite(MOTOR_A_PIN, HIGH);
420         digitalWrite(MOTOR_B_PIN, LOW);
421         break;
422     case 2:
423         digitalWrite(MOTOR_A_PIN, LOW);
424         //digitalWrite(MOTOR_B_PIN, LOW);    // FOR DEBUGGING.
425         digitalWrite(MOTOR_B_PIN, HIGH);
426         break;
427 }
428
429 Drive();    // The driving and collision avoidance algorithm
430 }

```


431

INTENTIONALLY BLANK

4.2 obstacle-challenge.ino

4.2.1 Introduction to the code

We strongly recommend that you first study the code of the Open Challenge (Chapter 4.1) to understand how the different functions work and then come to this chapter, because in order not to unnecessarily repeat the explanations we will simplify those already explained.

4.2.2 Code structure

All the code for this challenge is contained in a single .ino file, named obstacle-challenge.ino. This code is executed in the robot's ESP32 main.

There are no other programs for this challenge.

We have not created any additional libraries or files as required.

4.2.3 Algorithm design

A flowchart of the code's operation can be seen below (fig. 4.3).

As can be seen, as soon as the robot is powered on, it enters standby mode until the Start button is pressed. Meanwhile, as observed in the source code, the robot's various systems and peripherals are initialized (HuskyLens 2, I2C communication, color sensor, servomotor, etc.).

Once the Start button is pressed, the robot enters the loop. We can divide the loop's operation into three parts:

1. Robot Movement
2. Limit Case Checking
3. Camera analysis

4.2.3.1 Robot movement

The basic movement strategy is the same as in the Open Challenge: use the side TF-Luna sensors to keep the robot as centered as possible between the side walls of the track. In fact, if the Obstacle Challenge were to occur without any traffic signs, the robot would complete the challenge in the same way as in the Open Challenge.

Building on the foundation already explained, the sign avoidance mechanism is added. This involves using information received from the camera (block type, position, etc.) and the block's X-axis position to add a correction to the servo's rotation angle, depending on the block's color. For example, if a green block is encountered slightly to the right of the robot, a slight servo offset

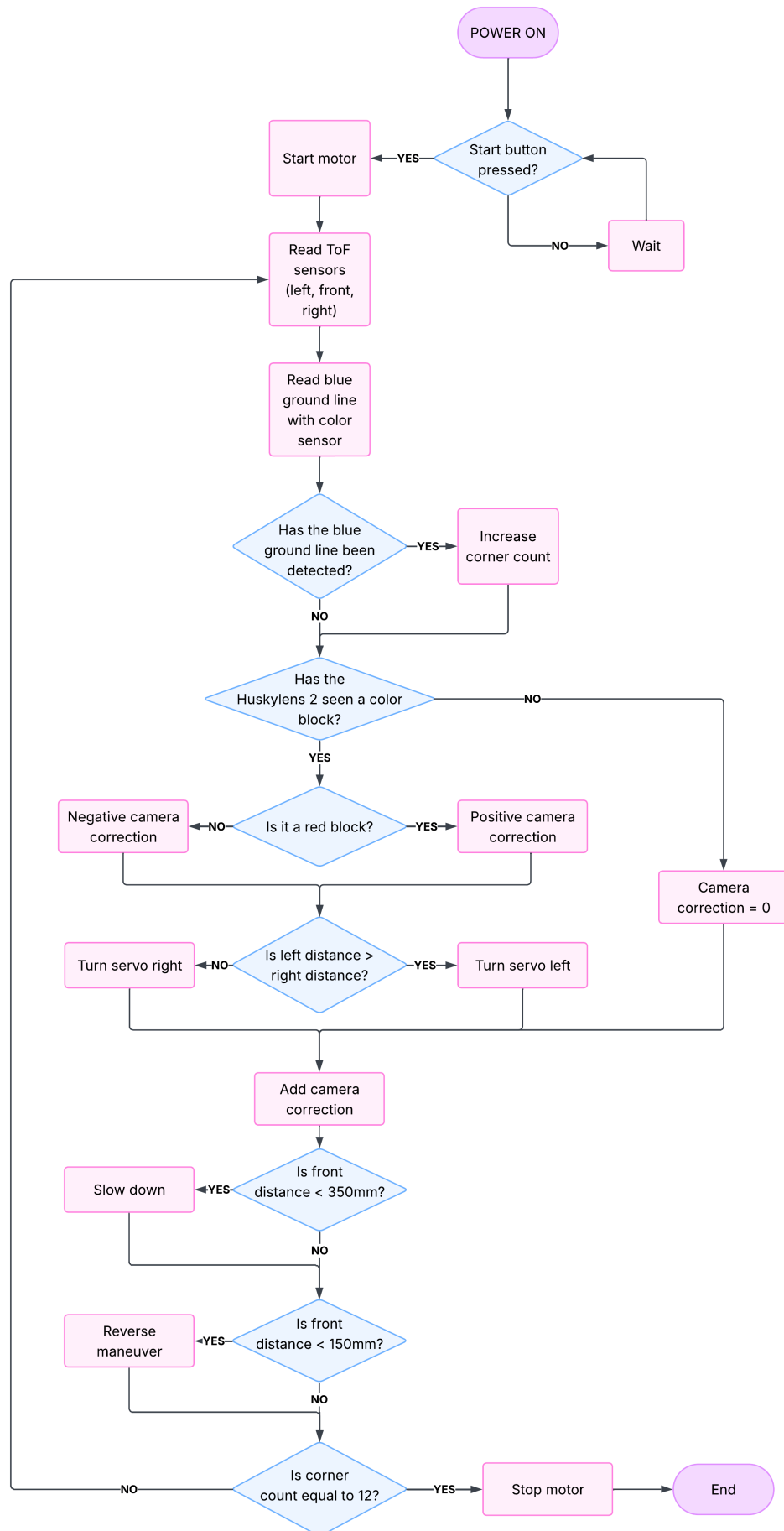


Figure 4.3: Obstacle challenge algorithm flowchart

will be added so that the wheels turn a little more to the left than they would if there were no block. Conversely, if a red block is encountered far to the right of the robot, a large offset will be added so that the robot can avoid the blocks.

The basic strategy was already tested during the Open Challenge trials. The obstacle section involved trial and error until it worked. The biggest problems we encountered were related to the block detection mechanism, which we explain below, rather than the dodge mechanism.

4.2.3.2 Check of limit cases

The loop also performs certain checks at all times on the following things, as shown in the flowchart:

- If the color sensor facing the ground detects the blue line, then the internal counter `corner_count`, which measures how many corners the robot has passed through, is incremented by one. This tells us how many sections the robot has traversed.
- If the front Luna ToF sensor measures a distance less than 350mm, the motor speed is reduced to avoid a collision. This can sometimes happen when getting too close to walls.
- If the front Luna ToF sensor measures a distance less than 150mm, a collision is practically imminent, and the reversing process is activated. This temporarily disables the robot's movement and makes it reverse to avoid getting stuck.
- If the `corner_counter` is equal to 12, the robot has already completed three laps and must stop in the final section.

4.2.3.3 Camera analysis

First, regarding the camera analysis, it's worth noting that the Huskylens 2 wasn't our first choice, as explained in the electronics and sensors section. However, introducing the Huskylens 2 allowed us to delegate some of the detection and calibration algorithms to those already integrated into the camera, thus improving the code's reliability and ease of use.

We utilize the Huskylens 2's color detection algorithm, as it detects blocks of color, which is precisely what we needed. The ESP32S3 constantly queries what the Huskylens 2 is seeing, obtaining certain data from each block it detects. Internally, we then select the largest block, which will be the closest to the camera. This way, if it detects two signals, one near and one far, it first acts on the nearer one.

Once the block is selected, we send its color and X position relative to the robot to the main code, so the movement algorithm can handle the rest.

To calibrate the camera to the colors, we used the color training functionality already available on the Huskylens 2, accessible via the screen and a button. In the various tests we conducted, we found that compared to the Huskylens 1 (which we also had at the academy), the Huskylens

2 is much better at distinguishing colors and seeing the same color in different lighting environments. This is ideal because we cannot guarantee that the lighting will be constant across the entire board on competition day, and this way, even if one part is brighter than another, it will see the correct color in all cases.

4.2.4 Bugs and errors found in the code

In terms of movement, since we based our work on the already functional Open Challenge code, we haven't encountered any particular bugs. We recommend reviewing that section to see what might be causing problems.

Some errors already related to this challenge include typical wiring mistakes, such as incorrectly connecting the TX and RX pins. We'd like to highlight these main issues that bothered us the most:

- One problem we observed after extensive testing is that the wheel servos wear out quite quickly, requiring frequent replacement. There's no single solution, as wear is common with any machine; however, adapting to this wear is key. We've decided to purchase several spare servos to take with us to competitions. Furthermore, we've learned that if we redesign the robot in the future, we should make servo replacement easier. Currently, it's a bit difficult, but we don't have the time to redesign the entire mechanism.
- Another issue is that we originally used an ESP32 S3 CAM. This option is incredibly cheaper than using the Huskylens 2; however, after using it for a while, we found that calibrating it for our challenge was really difficult, since we had to manually manage nearly 10 values for each color and then manually change them via code, which was very time-consuming. This, combined with the fact that the detection algorithm we set up wasn't very good, led us to opt for the Huskylens 2.

4.2.5 Functions

	Name
uint16_t	LunaRead16(uint8_t addr, uint8_t reg)
uint16_t	LunaDistance(uint8_t addr)
void	Drive()
bool	InitHuskyLens()
bool	RequestHuskyLens()
void	setup()
void	loop()

Table 4.4: Each function and its type

4.2.6 Constants

Table 4.5: Each variable and its type

	Name	Value
<code>const uint8_t</code>	<code>LUNA_COUNT</code> Number of Luna time of flight sensors	3
<code>const uint8_t[LUNA_COUNT]</code>	<code>LUNA_ADDR</code> I2C address of each ToF sensor. Right, Left, Front	0x11, 0x12, 0x13
<code>const uint16_t</code>	<code>LUNA_STRENGHT_MIN</code> Minimum strenght needed from the ToF sensor to accept a measurement	20
<code>const uint16_t</code>	<code>LUNA_CLOSE_STRENGTH</code> Minumum strength needed to consider a 0 distance measurement too close	150
<code>const uint16_t</code>	<code>LUNA_CLOSE_MM</code> Value (mm) used in case of a too close measurement	150
<code>const uint16_t</code>	<code>FRONT_BRAKE_MM</code> Progressive braking start distance (mm) threshold	350
<code>const uint16_t</code>	<code>FRONT_REAR_MM</code> Reverse gear start distance (mm) threshold	150
<code>const int</code>	<code>MOTOR_B_PIN</code>	20
<code>const int</code>	<code>MOTOR_A_PIN</code>	10
<code>const int</code>	<code>ENA_PIN</code>	15
<code>const int</code>	<code>BUTTON_PIN</code>	13
<code>const int</code>	<code>ODOMETER_PIN</code>	15
<code>const int</code>	<code>SDA_PIN</code>	11
<code>const int</code>	<code>SCL_PIN</code>	12
<code>const int</code>	<code>SDA_PIN_COLOR</code> SDA for TCS34725 and HuskyLens 2	40
<code>const int</code>	<code>SCL_PIN_COLOR</code> SCL for TCS34725 and HuskyLens 2	41
<code>const int</code>	<code>SERVO_PIN</code>	9
<code>const int</code>	<code>BASE_SPEED</code>	120
<code>const int</code>	<code>HUSKYLENS_ADDR</code> HuskyLens 2 I2C address	0x50

Continues on next page...

Table 4.5: Each variable and its type (continued)

<i>... comes from previous page</i>		
	Name	Value
const int	HUSKYLENS_TIMEOUT_MS Time it waits for HuskyLens response	1000
const int	HUSKYLENS_ID_GREEN_1 ID assigned to green in HuskyLens	1
const int	HUSKYLENS_ID_GREEN_2 ID assigned to green in HuskyLens	3
const int	HUSKYLENS_ID_RED_1 ID assigned to red in HuskyLens	2
const int	HUSKYLENS_ID_RED_2 ID assigned to red in HuskyLens	4
const int	CAM_CLOSE_AREA Block area (width*height px) that means the object is too close and triggers reverse	90000
const int	CAM_COUNT_NUMBER How many drive loops the camera correction has to be applied	15
const int	CAM_CLOSE_NUMBER How many drive loops the block reverse flag has to be up	8

4.2.7 Variables

Table 4.6: Each variable and its type

	Name
Adafruit_TCS34725	tcs Color sensor TCS34725 object.
Servo	servo Servo object.
int	button_state START button state.
int	corner_count Counts the number of corners the robot has passed. It should go from 0 to 12.
long	delay_line NOT IN USE.
<i>Continues on next page...</i>	

Table 4.6: Each variable and its type (continued)

... comes from previous page	
	Name
int	travel_sense Controls travel sense. 0 = BRAKE, 1 = FORWARD, 2 = BACKWARD.
byte	rear_phase Controls the rear phases. 0 = NORMAL, 1 = STOP, 2 = REVERSE, 3 = RESUME.
long	end_motor_millis Variable to control the additional time added from entering the last section until the motor stops.
long	prev_millis_line_detection Variable to control time and avoid multiple detections of the same line.
long	prev_millis_front Variable to control time and avoid multiple detections of the wall from the front sensor.
unsigned long	rear_phase_until Variable to control time for the reverse system.
long	odo_filter Variable to control time and avoid multiple detections of the same step of the odometer.
int	odo_integral Odometer step count.
int	d_servo How much the servo needs to move.

4.2.8 Functions documentation

4.2.8.1 function LunaRead16

```

1      uint16_t LunaRead16(uint8_t addr, uint8_t reg)
2

```

Parameters:

- **addr** I2C address of the ToF Luna that you wish to request
- **reg** Value to be requested. 0x00 for distance. 0x02 for strength

Return: Desired value of the ToF Luna in question

Request the demanded value of the indicated ToF Luna via I2C connectivity.

4.2.8.2 function LunaDistance

```
1      uint16_t LunaDistance(uint8_t addr)
2
```

Parameters:

- **addr** I2C address of the ToF Luna that you wish to request

Return:

- Distance in mm measured
- 0xFFFF if failed to measure

This function returns the distance in mm measured by the requested Luna ToF and also ensures that this measurement makes sense using the strength of the Luna ToF, thus giving consistent results in borderline cases.

4.2.8.3 function InitHuskyLens

```
1      bool InitHuskyLens()
2
```

Parameters: None

Return:

- true if initialization successful
- false if initialization was unsuccessful

Initialize HuskyLens communication (shares Wire1 with the TCS34725).

4.2.8.4 function RequestHuskyLens()

```
1      bool RequestHuskyLens()
2
```

Parameters: None

Return:

- true if a valid green/red block was detected
- false if otherwise

Request color and position data from HuskyLens and picks the largest detected color block and computes a servo correction.

4.2.8.5 function Drive

```
1 void Drive()  
2
```

Parameters: None

Return: void

The driving and collision avoidance algorithm.

4.2.8.6 function setup

```
1 void setup()  
2
```

Parameters: None

Return: void

This code only runs once when the ESP32 starts up. Initialize all systems and peripherals and wait for the start button to be pressed to start the program.

4.2.8.7 function loop

```
1 void loop()  
2
```

Parameters: None

Return: void

This code only runs all the time.

4.2.9 Source code

```

1 /*****
2  *   FILE NAME:   obstacle-challenge
3  *
4  *
5  *   PURPOSE:     This program is the version of the main
6  *                 robot code for the Obstacle Challenge of the
7  *                 Future Engineers category of the World
8  *                 Robot Olympiad.
9  *
10 *
11 *   HOW IT WORKS: The program uses side time-of-flight
12 *                 distance sensors to keep the robot
13 *                 constantly aligned with the center line
14 *                 between the two side walls. Additionally,
15 *                 in the event of a frontal collision or
16 *                 when approaching a collision, the front
17 *                 sensor is used to move backward or reduce
18 *                 speed, respectively. It is connected to a
19 *                 HuskyLens 2 that tells if there is an object,
20 *                 its position and color, so this code makes the
21 *                 servo turn enough to pass the obstacle through
22 *                 the right side.
23 *
24 *
25 *   LIBRARIES USED:
26 *   Name           Description
27 *   ----           -
28 *   ESP32Servo     Functions required to control a
29 *                   servomotor with the ESP32
30 *   Wire           Allows I2C communication
31 *   Adafruit_TCS34725 Functions required to use the
32 *                   Adafruit TCS34725 RGB Color Sensor
33 *   DFRobot_HuskyLensV2 Functions for HuskyLens AI vision
34 *                   sensor
35 *
36 *   OTHER PROGRAMS NEEDED TO WORK
37 *   Name           Description
38 *   ----           -
39 *   none
40 *
41 *
42 *   CREATED BY: Vila-Stem 8
43 *
44 *   LAST MODIFIED: August 31st 2026
45 *
46 *
47 *   REPOSITORY: https://github.com/Vila-Stem/
WRO_FUTURE_ENGINEERS_2026

```

```

48  *
49  ****
50  */
51
52
53  /* INCLUDE LIBRARIES */
54  #include <ESP32Servo.h>
55  #include <Wire.h>
56  #include <Adafruit_TCS34725.h>
57  #include "DFRobot_HuskyLensV2.h"
58
59
60  /* CONSTANT VALUES */
61  /* Related to the Luna time of flight (ToF) sensor */
62  const uint8_t LUNA_COUNT = 3;          ///<
        Number of Luna time of flight sensors
63  const uint8_t LUNA_ADDR[LUNA_COUNT] = {0x11, 0x12, 0x13};  ///< I2C
        address of each ToF sensor. Right, Left, Front
64  const uint16_t LUNA_STRENGTH_MIN = 20;  ///<
        Minimum strength needed from the ToF sensor to accept a
        measurement
65  const uint16_t LUNA_CLOSE_STRENGTH = 150;  ///<
        Minimum strength needed to consider a 0 distance measurement too
        close
66  const uint16_t LUNA_CLOSE_MM = 150;  ///<
        Value (mm) used in case of a too close measurement
67
68  /* Collision avoidance threshold distances */
69  const uint16_t FRONT_BRAKE_MM = 350;  ///<
        Progressive braking start distance (mm) threshold
70  const uint16_t FRONT_REAR_MM = 150;  ///<
        Reverse gear start distance (mm) threshold
71
72  /* Pins */
73  const int MOTOR_B_PIN = 20;
74  const int MOTOR_A_PIN = 10;
75  const int ENA_PIN = 15;  ///<
        Enabled motor pin. Used with PWM for speed control
76  const int BUTTON_PIN = 13;  ///<
        START button pin
77  const int SDA_PIN = 11;
78  const int SCL_PIN = 12;
79  const int SDA_PIN_COLOR = 40;  ///< SDA
        for TCS34725 and HuskyLens 2
80  const int SCL_PIN_COLOR = 41;  ///< SCL
        for TCS34725 and HuskyLens 2
81  const int SERVO_PIN = 9;
82
83  const int BASE_SPEED = 120;
84
85  /* HuskyLens */

```

```

86 const int HUSKYLENS_ADDR = 0x50;          ///<
    HuskyLens 2 I2C address
87 const int HUSKYLENS_TIMEOUT_MS = 1000;    ///<
    Time it waits for HuskyLens response
88 const int HUSKYLENS_ID_GREEN_1 = 1;        ///< ID
    assigned to green in HuskyLens
89 const int HUSKYLENS_ID_GREEN_2 = 3;        ///< ID
    assigned to green in HuskyLens
90 const int HUSKYLENS_ID_RED_1 = 2;          ///< ID
    assigned to red in HuskyLens
91 const int HUSKYLENS_ID_RED_2 = 4;          ///< ID
    assigned to red in HuskyLens
92 const int CAM_CLOSE_AREA = 90000;          ///<
    Block area (width*height px) that means the object is too close
    and triggers reverse
93 const int CAM_COUNT_NUMBER = 15;           ///< How
    many drive loops the camera correction has to be applied
94 const int CAM_CLOSE_NUMBER = 8;            ///< How
    many drive loops the block reverse flag has to be up
95 // ALGORITHM_COLOR_RECOGNITION is defined by the HuskyLens library
96 // (value 4). This guard is only a safety net for library versions
97 // that do not expose it.
98 #ifndef ALGORITHM_COLOR_RECOGNITION
99 #define ALGORITHM_COLOR_RECOGNITION 4
100 #endif
101
102
103 /* CREATE OBJECTS */
104 Adafruit_TCS34725 tcs = Adafruit_TCS34725(
    TCS34725_INTEGRATIONTIME_50MS, TCS34725_GAIN_60X);
105 Servo servo;
106 HuskylensV2 huskylens;
107
108
109 /* VARIABLES */
110 int button_state = HIGH;                   ///< START button state
111 int corner_count = 0;                      ///< Counts the number of
    corners the robot has passed. It should go from 0 to 12.
112 long delay_line = 0;                      ///< NOT IN USE
113 int travel_sense = 1;                     ///< Controls travel sense. 0
    = BRAKE, 1 = FORWARD, 2 = BACKWARD
114 byte rear_phase = 0;                      ///< Controls the rear phases.
    0 = NORMAL, 1 = STOP, 2 = REVERSE, 3 = RESUME
115 long end_motor_millis = 0;                ///< Variable to control the
    additional time added from entering the last section until the
    motor stops
116 long prev_millis_line_detection = 0;      ///< Variable to control time
    and avoid multiple detections of the same line
117 long prev_millis_front = 0;               ///< Variable to control time
    and avoid multiple detections of the wall from the front sensor
118 unsigned long rear_phase_until = 0;       ///< Variable to control time

```

```

    for the reverse system
119 int d_servo = 0;          ///< How much the servo needs
    to move
120 unsigned long last_huskylens_request = 0; ///< Variable to control
    time for calling the Huskylens 2
121 int cam_correction = 0;    ///< Camera correction to be
    applied to the servo
122 int cam_count = 0;        ///< Controls how many drive
    loops the camera correction has been applied
123 int cam_close = 0;        ///< Controls how many drive
    loops the block reverse flag is up
124
125
126 /* FUNCTIONS */
127
128 /**
129  * Request the demanded value of the indicated ToF Luna via I2C
    connectivity.
130  * @param addr I2C address of the ToF Luna that you wish to request
131  * @param reg Value to be requested. 0x00 for distance. 0x02 for
    strength
132  * @return Desired value of the ToF Luna in question
133  * @see LunaDistance()
134  */
135 uint16_t LunaRead16(uint8_t addr, uint8_t reg) {
136     Wire.beginTransmission(addr);          // Begins
        I2C communication with the given address
137     Wire.write(reg);                      // Tells ToF
        Luna that we desire the given type of information
138     Wire.endTransmission(false);          // Keeps the
        I2C communication active with the ToF Luna
139     Wire.requestFrom(addr, (uint8_t)2);    // Requests
        2 bytes data from the ToF Luna. It releases the I2C bus after the
        request.
140     if (Wire.available() < 2) return 0xFFFF; // If less
        than 2 bytes retrieved returns 0xFFFF as failed
141     return (uint16_t)(Wire.read() | (Wire.read() << 8)); // Returns
        the data
142 }
143
144
145 /**
146  * This function returns the distance in mm measured by the
    requested
147  * Luna ToF and also ensures that this measurement makes sense using
148  * the strength of the Luna ToF, thus giving consistent results in
149  * borderline cases.
150  * @param addr I2C address of the ToF Luna that you wish to request
151  * @return Distance in mm measured
152  * @return 0xFFFF if failed to measure
153  * @see LunaRead16()

```

```

154 */
155 uint16_t LunaDistance(uint8_t addr) {
156     uint16_t d = LunaRead16(addr, 0x00);           // Request ToF Luna
157     distance
158     uint16_t strength = LunaRead16(addr, 0x02);    // Request ToF Luna
159     strength
160     if (d == 0xFFFF || strength == 0xFFFF || strength <
161         LUNA_STRENGTH_MIN) return 0xFFFF; // Returns 0xFFFF if failed
162     if (d == 0) // The distance may
163         be 0 because the sensor is too close to a wall and is saturated,
164         or because it measures infinity
165     {
166         if (strength >= LUNA_CLOSE_STRENGTH) return LUNA_CLOSE_MM; //
167         If the strength is very high, the sensor is too close to the wall
168         .
169         return 1500; //
170         If the strength is very low, the sensor measures infinity.
171     }
172     return d*10; // The ToF Luna returns cm, but the code needs mm
173 }
174
175 /**
176  * Initialize HuskyLens communication (shares Wire1 with the
177  * TCS34725).
178  * @param none
179  * @return true if initialization successful
180  * @return false if initialization was unseccessful
181  * @see RequestHuskyLens()
182  */
183 bool InitHuskyLens() {
184     Wire1.setClock(100000); // Wire1 is already initialized in setup
185     () for the Adafruit TCS34275
186     // Debug code in case it is not working the Huskylens 2
187     if (!huskylens.begin(Wire1)) {
188         Serial.println("ERROR: HuskyLens not detected on I2C (Wire1)");
189         return false;
190     }
191
192     // Set algorithm to Color Recognition
193     if (!huskylens.switchAlgorithm((eAlgorithm_t)
194         ALGORITHM_COLOR_RECOGNITION)) {
195         Serial.println("ERROR: Failed to set Color Recognition algorithm
196         ");
197         return false;
198     }
199
200     Serial.println("HuskyLens initialized successfully");
201     return true;
202 }

```

```

193
194 /**
195  * Request color and position data from HuskyLens.
196  * and picks the largest detected color block and
197  * computes a servo correction.
198  * @param none
199  * @return true if a valid green/red block was detected
200  * @return false if otherwise
201  * @see InitHuskyLens()
202  */
203 bool RequestHuskyLens() {
204     // Refresh the result cache from the sensor
205     huskylens.getResult((eAlgorithm_t)ALGORITHM_COLOR_RECOGNITION);
206
207     // Check if there are results available for the color recognition
208     // algorithm
209     if (!huskylens.available((eAlgorithm_t)ALGORITHM_COLOR_RECOGNITION
210 ))
211     {
212         return false;
213     }
214
215     // How many color objects are being detected
216     int count = huskylens.getCachedResultNum((eAlgorithm_t)
217         ALGORITHM_COLOR_RECOGNITION);
218     if (count <= 0)
219     {
220         return false;
221     }
222
223     // Find the largest block (the closest one)
224     int bestIdx = -1;
225     int maxArea = 0;
226     for (int i = 0; i < count; i++)
227     {
228         Result* result = huskylens.getCachedResultByIndex((eAlgorithm_t)
229             ALGORITHM_COLOR_RECOGNITION, i); // Gets the result data of each
230         // block by calling it by its ID number
231         if (result)
232         {
233             int area = result->width * result->height; //
234             // Calculates the block area (width * height)
235             if (area > maxArea) //
236             // Checks if the area of this block is bigger than the max. area
237             {
238                 maxArea = area; //
239                 // Saves the biggest area detected
240                 bestIdx = i; //
241                 // Saves the ID number of the biggest area detected
242             }
243         }
244     }

```



```

235 }
236 if (bestIdx == -1) //
    Checks that the "find largest block" system did not failed
237 {
238     return false;
239 }
240
241 // Gets the result data of the biggest area block
242 Result* result = huskylens.getCachedResultByIndex((eAlgorithm_t)
    ALGORITHM_COLOR_RECOGNITION, bestIdx);
243 if (!result)
244 {
245     return false;
246 }
247
248 // Map HuskyLens learned color ID to our color scheme (1 = GREEN,
    2 = RED)
249 int color = 0;
250 if (result->ID == HUSKYLENS_ID_GREEN_1 || result->ID ==
    HUSKYLENS_ID_GREEN_2 )
251 {
252     color = 1; // GREEN
253 }
254 else if (result->ID == HUSKYLENS_ID_RED_1 || result->ID ==
    HUSKYLENS_ID_RED_2)
255 {
256     color = 2; // RED
257 }
258 else
259 {
260     return false; // Unknown color. Failed.
261 }
262
263 // HuskyLens returns x center (0 left side of the camera screen,
    320 right side of the camera screen)
264 int cx = result->xCenter;
265 int area = result->width * result->height;
266 if (cx <= 0 || cx > 320) // Validate detection making sure it
    is between the allowed values
267 {
268     return false;
269 }
270
271 // Calculate correction
272 if (color == 1) // GREEN, negative correction to go left
273 {
274     cam_correction = -((320 - cx) / 6);
275 }
276 else if (color == 2) // RED, positive correction to go right
277 { // RED
278     cam_correction = cx / 6;

```

```

279 }
280
281 cam_count = CAM_COUNT_NUMBER;
282
283 // Detects if a block is too close to the camera and the ToF Luna
    frontal sensor has not seen it.
284 // It uses the area of the block to know if it is too big and has
    to start the reverse manouver.
285 if (area >= CAM_CLOSE_AREA)
286 {
287     cam_close = CAM_CLOSE_NUMBER;
288 }
289
290 // Debug output
291 const char* colorName = (color == 1) ? "GREEN" : "RED";
292 Serial.printf("HUSKYLENS: color=%s cx=%d area=%d corr=%d\n",
    colorName, cx, area, cam_correction);
293
294 // Returns true if everything worked properly
295 return true;
296 }
297
298
299 /**
300  * The driving, collision avoidance and obstacle-passing mechanism.
301  * @see LunaDistance()
302  * @see RequestHuskyLens()
303  */
304 void Drive() {
305
306     // Initialize variable for reading the ToF sensors
307     static signed int distance_right, distance_left, distance_front;
308     uint16_t d;
309
310     //debug
311     distance_front = 1000000000;
312
313     // Initialize variables for the camera information
314     static int count_correction = 0;           // How many drive loops
        the camera correction has been applied
315     static signed int correction = 0;         // Correction applied to
        the servo
316
317     // Request HuskyLens information every 100 ms
318     if (millis() - last_huskylens_request > 100)
319     {
320         RequestHuskyLens();
321         last_huskylens_request = millis();
322     }
323
324     // It executes only if a color block has been detected

```

```

325 // Saves the RequestHuskyLens() variables to local variables to
    this fuction
326 if (cam_count > 0)
327 {
328     count_correction = cam_count;
329     correction = cam_correction;
330     cam_count = 0;
331 }
332
333 // These lines read the ToF Luna sensors and save the distance
334 // in the corresponding variable as long as it hasn't failed
    reading.
335 d = LunaDistance(LUNA_ADDR[0]);
    // Right distance sensor
336 if (d != 0xFFFF)
337 {
338     distance_right = d;
339     if (distance_right > 1500) distance_right = 1500;
    // Limit to 1500mm
340 }
341 d = LunaDistance(LUNA_ADDR[1]);
    // Left distance sensor
342 if (d != 0xFFFF)
343 {
344     distance_left = d;
345     if (distance_left > 1500) distance_left = 1500;
    // Limit to 1500mm
346 }
347 /*d = LunaDistance(LUNA_ADDR[2]);
    // Front distance sensor
348 if (d != 0xFFFF)
349 {
350     distance_front = d;
351 }*/
352
353 // If front sensor or Huskylens 2 detects an obstacle init rear
    sequence. This sequence is time based
354 if (distance_front < FRONT_REAR_MM || cam_close > 0)
355 {
356     if (rear_phase == 0)
357     {
358         if ((prev_millis_front + 1200) <= millis()) // This if
            statement avoids multiple detection of the same wall
359         {
360             prev_millis_front = millis();
361             rear_phase = 1;
362             rear_phase_until = millis() + 250;
363         }
364     }
365 }
366

```

```

367 // Decay the camera "close" flag while the obstacle passes away
368 if (cam_close > 0) cam_close--;
369
370 // Rear sequence. This is time based
371 if (rear_phase >= 1)
372 {
373     switch (rear_phase)
374     {
375         case 1:                // Stops the car for 250 ms (by default)
376             travel_sense=0;
377             if (millis() >= rear_phase_until){ rear_phase=2;
rear_phase_until = millis() + 1500; }
378             break;
379         case 2:
380             travel_sense=2;    // Reverse the car for 1500 ms (by default
381                                )
382             if (millis() >= rear_phase_until){ rear_phase=3;
rear_phase_until = millis() + 500; }
383             break;
384         case 3:                // The robot continues forward.
385             travel_sense=1;
386             rear_phase=0;
387             break;
388     }
389
390 // The servo needs to move a certain amount depending on the
391 // difference of distance measured between the left and
392 // the right ToF Luna sensors
393 d_servo = (distance_right - distance_left)/10;
394
395 // Limits the amount a servo can move in order to avoid collision
396 // of the wheels and the frame
397 if (d_servo > 39) d_servo = 39;
398 if (d_servo < -39) d_servo = -39;
399
400 if (count_correction > 0)
401 {
402     d_servo = correction;
403     count_correction--;
404 }
405
406 // In case of reverse, limited steering to avoid flipping
407 if (rear_phase == 2)
408 {
409     //if (d_servo > 40) d_servo = 12;
410     //if (d_servo < -12) d_servo = -12;
411     servo.write(-d_servo + 90);
412 }
413 else if (travel_sense == 1)
414 {

```

```

414     servo.write(d_servo + 90);
415 }
416
417 // Speed control
418 int base_speed = BASE_SPEED;           // Base speed of
    the robot
419
420 if (distance_front < FRONT_REAR_MM || cam_close > 0) // Reduces
    the speed before starting the reverse manoeuvre
421 {
422     base_speed = 120;
423 }
424 else if (distance_front < FRONT_BRAKE_MM)           // Reduces the
    speed when approaching a frontal wall
425 {
426     base_speed = 120;
427 }
428
429 if (d_servo >= 0)
430 {
431     ledcWrite(ENA_PIN, base_speed + d_servo);       // Sends the motor
    the speed. It is increased when turning
432 }
433 else
434 {
435     ledcWrite(ENA_PIN, base_speed - d_servo);       // Sends the motor
    the speed. It is increased when turning
436 }
437 }
438
439
440 /* SETUP AND LOOP FUNCTIONS */
441
442 /**
443  * This code only runs once when the ESP32 starts up.
444  * Initialize all systems and peripherals and wait for the
445  * start button to be pressed to start the program.
446  * @param none
447  * @return void
448  */
449 void setup() {
450     Serial.begin(115200);           // Starts Serial
    for debugging reasons
451     Wire.begin(SDA_PIN, SCL_PIN);   // I2C
    communication for ToF Luna
452     Wire.setTimeout(100);
453     Wire.setClock(100000);          // ToF Luna at 100
    kHz
454     Wire1.begin(SDA_PIN_COLOR, SCL_PIN_COLOR);      // I2C
    communication for TCS34725 color sensor and Huskylens 2
455

```

```

456  servo.attach(SERVO_PIN);                                // Initialize
    Servo
457  servo.write(90);                                        // 0 position
    Servo
458
459  // Pin Modes
460  pinMode(BUTTON_PIN, INPUT_PULLUP);
461  pinMode(MOTOR_A_PIN, OUTPUT);
462  pinMode(MOTOR_B_PIN, OUTPUT);
463
464  // Stop the motor
465  digitalWrite(MOTOR_A_PIN, LOW);
466  digitalWrite(MOTOR_B_PIN, LOW);
467
468  ledcAttachChannel(ENA_PIN, 1000, 8, 2);                // PWM for the
    motor. Frequency = 1000, Resolution = 8, Channel = 2
469
470  // FOR DEBUGGING
471  // Checks ToF Luna presence and debug which ones don't work
    through the serial port.
472  for (uint8_t i = 0; i < LUNA_COUNT; i++)
473  {
474      Wire.beginTransaction(LUNA_ADDR[i]);
475      if (Wire.endTransmission() == 0)
476      {
477          Serial.print("TF-Luna ");
478          Serial.print(i);
479          Serial.print(" OK (0x");
480          Serial.print(LUNA_ADDR[i], HEX);
481          Serial.println(")");
482      }
483      else
484      {
485          Serial.print("TF-Luna ");
486          Serial.print(i);
487          Serial.print(" NOT found (0x");
488          Serial.print(LUNA_ADDR[i], HEX);
489          Serial.println(")");
490      }
491  }
492
493  // Initialize color sensor. DEBUG through the serial port if not
    working.
494  if (tcs.begin(0x29, &Wire1))
495  {
496      Serial.println("Found TCS34725 sensor");
497  }
498  else
499  {
500      Serial.println("WARNING: TCS34725 not detected, continuing
        without color.");

```

```

501 }
502
503 // Initialize HuskyLens
504 if (!InitHuskyLens())
505 {
506     Serial.println("WARNING: HuskyLens init failed, continuing
507         without vision.");
508 }
509
510 // Waits for START button to be pressed
511 int countButton = 0;
512 while (countButton <= 10)
513 {
514     button_state = digitalRead(BUTTON_PIN);
515     if (button_state == LOW) countButton++;
516     delay(10);
517 }
518
519 /**
520  * This code only runs all the time.
521  * @param none
522  * @return void
523  */
524 void loop() {
525     // Gets color temp from the Adafruit TCS34725 color sensor
526     uint16_t r, g, b, c, colorTemp; //
527     // Initialize needed variables
528     tcs.getRawData(&r, &g, &b, &c); //
529     // Gets the RGB and Clear channels
530     colorTemp = tcs.calculateColorTemperature_dn40(r, g, b, c); //
531     // Calculates the color temperature from the RGBC channels
532
533     // Increases corner count using the lines on the ground and the
534     // TCS34725 color sensor.
535     if (colorTemp > 3000)
536     {
537         if ((prev_millis_line_detection + 700) <= millis())
538         {
539             prev_millis_line_detection = millis();
540             corner_count++;
541         }
542     }
543
544     // Stops the car when the number of corners is reached (12 corners
545     // )
546     if (corner_count >= 12)
547     {
548         if ((end_motor_millis + 1500) <= millis())
549         {

```

```
546     travel_sense = 0;
547   }
548 }
549 else
550 {
551     end_motor_millis = millis();
552 }
553
554 // Sets the travel sense. 0 = BRAKE, 1 = FORWARD, 2 = BACKWARD
555 switch(travel_sense) {
556     case 0:
557         digitalWrite(MOTOR_A_PIN, LOW);
558         digitalWrite(MOTOR_B_PIN, LOW);
559         break;
560     case 1:
561         digitalWrite(MOTOR_A_PIN, HIGH);
562         //digitalWrite(MOTOR_A_PIN, LOW);           // FOR DEBUGGING.
563         digitalWrite(MOTOR_B_PIN, LOW);
564         break;
565     case 2:
566         digitalWrite(MOTOR_A_PIN, LOW);
567         digitalWrite(MOTOR_B_PIN, HIGH);
568         //digitalWrite(MOTOR_B_PIN, LOW);           // FOR DEBUGGING.
569         break;
570 }
571
572 Drive();           // The driving and collision avoidance algorithm
573 }
574
```


Photos

5.1 Robot photos

Several photos of our robot from different angles.

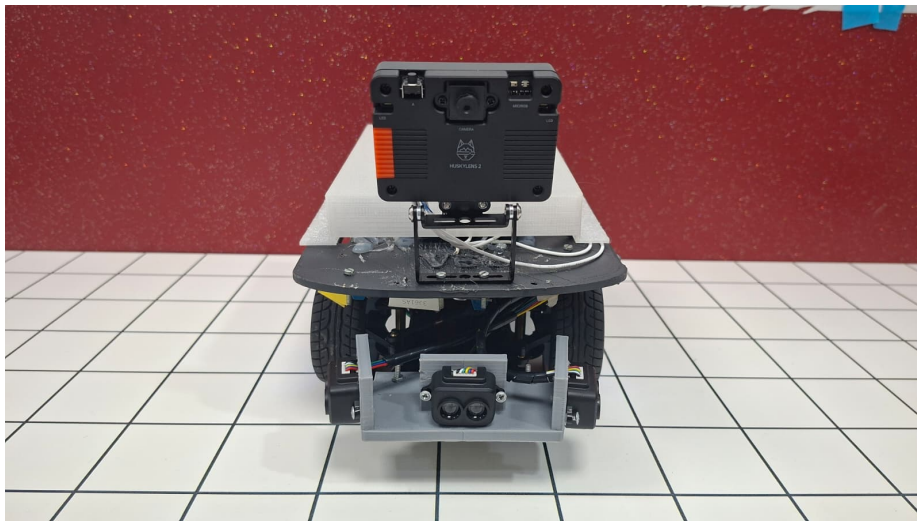


Figure 5.1: Front image

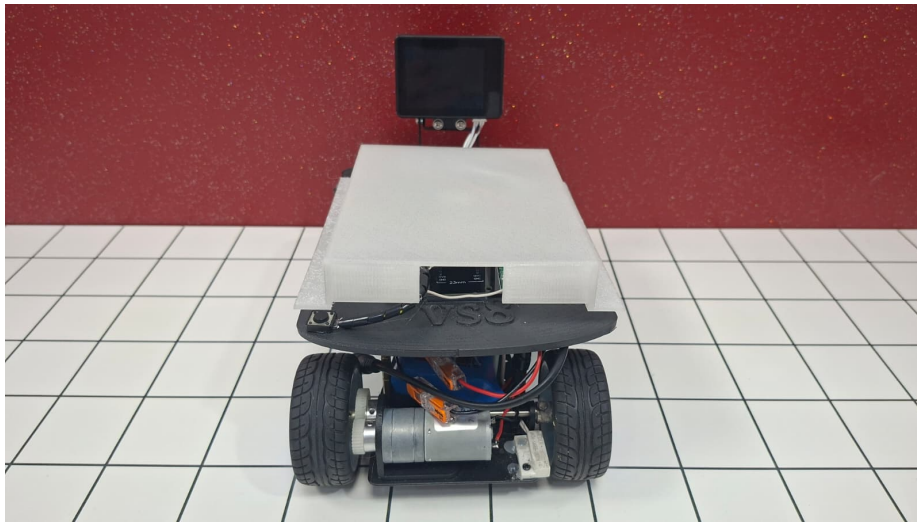


Figure 5.2: Back image



Figure 5.3: Left image



Figure 5.4: Right image



Figure 5.5: Top image

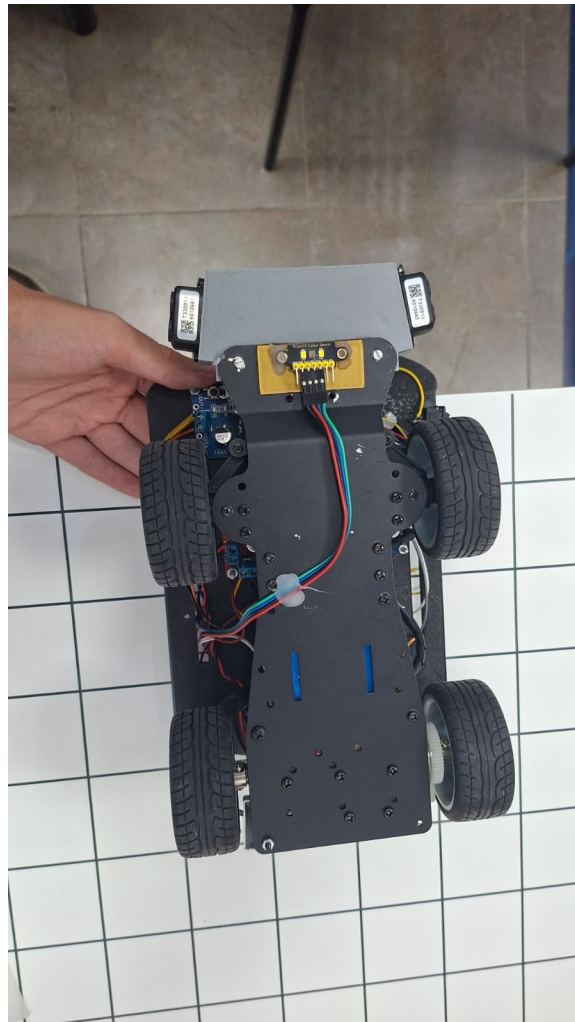


Figure 5.6: Bottom image

5.2 Team photos

Two photographs of the team.



Figure 5.7: Team photo with a LEGO-like style



Figure 5.8: Team photo at our academy

6

CHAPTER

Videos

We have uploaded two videos showing our robot completing each challenge. You can access them via the following links.

- **Open challenge:** <https://youtu.be/gIDQgGI8u58>
- **Obstacle challenge:** <http://www.example.com>